

REACT HOOKS RESEARCH REPORT

- * Topic: The Most Commonly Used Hooks in React with Detailed Explanation
- * Prepared for: Enactus IT Department '26
- * Submitted by: [Your Name]
- * Date: June 15, 2026

1. INTRODUCTION

React Hooks were introduced in React version 16.8. They completely revolutionized how front-end developers build applications by allowing them to use state management and other core React features (like lifecycle methods) inside functional components, entirely eliminating the need to write complex class components. This research report details the most essential and widely used hooks in modern React architecture, providing clear explanations and actual code implementation scenarios for each.

2. USESTATE

* Detailed Explanation:

useState is the most foundational hook in React. It allows functional components to declare, maintain, and update local state variables. It accepts the initial state as an argument and returns an array with exactly two elements: the current state value and a setter function used to update that value asynchronously.

* Use Case Scenarios:

Managing form input data, toggling visibility states (modals, dropdowns), controlling counters, and tracking user interactions.

* Code Example:

```
import React, { useState } from 'react';

function CounterExample() {
  const [count, setCount] = useState(0);

  return (
    <div style={{ padding: '20px', textAlign: 'center' }}>
      <p>Current Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
```

```
);  
}
```

3. USEEFFECT

* Detailed Explanation:

useEffect serves the purpose of executing side effects inside functional components. Side effects are actions that interact with systems outside the pure scope of React component rendering. It consolidates the classic class component lifecycle methods (componentDidMount, componentDidUpdate, and componentWillUnmount) into a single unified API.

* The Dependency Array Behaviour:

- No Array Provided: The effect runs on the initial render and after every re-render.
- Empty Array ([]): The effect runs exactly once when the component builds/mounts.
- Array with Variables ([prop, state]): The effect runs on mount and triggers again only when the specified variables change value.

* Use Case Scenarios:

Fetching data from an external REST API, configuring WebSocket connections, and manually mutating the browser DOM.

* Code Example:

```
import React, { useState, useEffect } from 'react';  
  
function DataFetchingExample() {  
  const [users, setUsers] = useState([]);  
  
  useEffect(() => {  
    fetch('https://jsonplaceholder.typicode.com/users')  
      .then(response => response.json())  
      .then(data => setUsers(data));  
  }, []);  
  
  return (  
    <ul>  
      {users.map(user => <li key={user.id}>{user.name}</li>)}  
    </ul>  
  );  
}
```

4. USECONTEXT

* Detailed Explanation:

useContext provides a clean, native way to subscribe to React Context. It enables components to access global data seamlessly across the entire application tree without relying on "prop drilling" (the tedious process of passing props through multi-layered child components that do not actually need the data).

* Use Case Scenarios:

Storing global state configs like theme tracking (Light/Dark mode layouts), localized multi-language preferences, and authenticated user session profiles.

* Code Example:

```
import React, { createContext, useContext } from 'react';
```

```
const ThemeContext = createContext('dark');
```

```
function App() {  
  return (  
    <ThemeContext.Provider value="dark">  
      <Toolbar />  
    </ThemeContext.Provider>  
  );  
}
```

```
function Toolbar() {  
  return <ThemedButton />;  
}
```

```
function ThemedButton() {  
  const theme = useContext(ThemeContext);  
  return <button className={`btn-${theme}`}>Applied Theme: {theme}</button>;  
}
```

5. USEREF

* Detailed Explanation:

useRef returns a mutable reference object whose .current property initializes with the argument passed to the hook. Key features of useRef include: mutating the .current property does not cause a component re-render, and it keeps values intact across renders. It is heavily utilized to persist reference markers directly to real DOM elements.

* Use Case Scenarios:

Programmatically focusing text input boxes, measuring DOM element dimensions, controlling HTML5 video playback actions, and tracking previous state values.

* Code Example:

```
import React, { useRef } from 'react';

function TextInputFocusExample() {
  const inputRef = useRef(null);

  const triggerFocus = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Click button to focus me..." />
      <button onClick={triggerFocus}>Focus Input Box</button>
    </div>
  );
}
```

6. USEMEMO

* Detailed Explanation:

useMemo is a dedicated performance optimization hook designed to return a memoized value. It caches the structural output of an expensive computation script between renders. React will bypass recalculating that value on future re-renders unless one of its declared tracked dependencies changes.

* Use Case Scenarios:

Filtering or sorting huge data arrays, parsing large JSON blocks, or performing heavy algorithmic equations inside components.

* Code Example:

```
import React, { useState, useMemo } from 'react';

function MathCalculationExample({ number }) {
  const factorial = useMemo(() => {
    let result = 1;
    for (let i = 1; i <= number; i++) result *= i;
    return result;
  }, [number]);
```

```
    return <div>Factorial Value: {factorial}</div>;  
  }  
}
```

7. USECALLBACK

* Detailed Explanation:

Conceptually similar to `useMemo`, `useCallback` focuses strictly on performance tuning, but instead of caching data outputs, it returns a memoized instance of a callback function itself. It guarantees that the functional reference remains completely identical across renders unless its matching dependency array undergoes a state shift.

* Use Case Scenarios:

Passing optimized event callbacks to memoized child components (`React.memo`) to avoid unnecessary re-render pipelines caused by reference disparities.

* Code Example:

```
import React, { useState, useCallback } from 'react';  
  
function ParentContainer() {  
  const [items, setItems] = useState([]);  
  
  const handleClear = useCallback(() => {  
    setItems([]);  
  }, []);  
  
  return <ChildResetButton onReset={handleClear} />;  
}
```

8. CONCLUSION

React Hooks represent an essential structural paradigm shift in front-end development patterns. Mastering basic hooks like `useState` and `useEffect` is vital for managing fluid runtime application data and asynchronous network logic, whereas incorporating performance hooks like `useMemo`, `useCallback`, and structural tools like `useRef` provides developers with complete precision over execution runtime and structural optimization.